STATESPACE FRAMEWORK

MICHAEL ZARGHAM

1. Event Based State Space Model

The core of our computer aided design tools contain discrete time differential equation in a state space. The following definitions are used to characterize the unbounded agent dimensional state space and to relate that bounded state space over which system requirements may be defined. This model does not require that the system be implemented with a centralized state but mathematics characterizes the macro scale evolution of the system in terms of formally defined micro elements. This framework is critical for understanding the relationships between agents and accounts within a decentralized system.

Furthermore, timescale of the state space model is defined so that agent actions action can be properly modeled. A discretization of the sequence and aggregation of actions into transaction blocks is necessary to simulate the system at this level. However, the construct provided is sufficiently general that if the discrete time is mapped to atomic events, the differential equations models may be used for event based simulation without deriving new models.

This model is defined over addresses. All addresses are defined by public key private key pairs, where private keys are used for cryptographic proof of the right to act as agent. Without loss of generality more complex schemes such as multi-signature schemes can be substituted for the simple private key proofs assumed here. Further characterization of cryptographic schemes is not required as this section focuses on the system dynamics rather than the means of enforcing those dynamics.

Definition 1. *Let $\mathcal{A}$ be the set of all possible* ***Addresses*** *as determined by the range of the cryptographic hash function used in the system implementation. At any time the set of addresses that exist is $\mathbf{A} \subset \mathcal{A}$.*

An address $a \in \mathbf{A}$ is called an agent when referring to the address which takes an action. An address $a \in \mathbf{A}$ is called an account when it contains code declaring one or more states and associated mechanisms which are exposed to other agents. All accounts are agents, but not all agents are accounts.

Definition 2. *The* ***Ledger State*** *is a shared data structure $\mathbf{L} \in \mathcal{L}$ which evolves when agents perform transactions by taking actions with respect to mechanisms; $\mathcal{L}$ denotes the space of all valid ledger states $\mathbf{L}$. The state $\mathbf{L} = \{\mathbf{X}, \mathbf{T}\}$ where T is a (partially) ordered*

Date: Sept 2018.

list of transactions. Each $t \in \mathbf{T}$ *is defined by an agent* $a \in \mathbf{A}$*, a mechanism* $f \in \mathcal{F}$ *and an action* $u \in \mathcal{U}$*.*

The list of transactions $\mathbf{T}$ need not be strictly ordered because transactions are local operations on the state with respect to accounts. If it suffices to know the order of any transactions with dependence on or modification of a shared element of the state $x \in \mathbf{X}$. Further formalization of event orderings is deferred from discussion in this document.

Definition 3. *Consider the* ***Local State*** *to be a subset of* X_i *to be subset of the global state* $\mathbf{X}$ *which is declared by account* $i \in \mathbf{A}$*. The address* i *is both agent and account so it has local state* X_{ii} *but other agents may control elements of* X_i*. The elements of the state declared by* $i \in \mathbf{A}$ *but controlled by* $j \in \mathbf{A}$ *are* X_{ij}*.*

It is immediate that each local state X_i must be declared by a unique account $i \in \mathbf{A}$. Therefore, the local states can be interpreted as a partition of $\mathbf{X}$ over accounts. It is not, however, assumed that there is a partition over agents dimension.

Definition 4. *Consider the set of* ***Mechanisms*** *to be* $\mathcal{F}$ *such that any* $f \in \mathcal{F}$ *is an operator*

$$f : \mathcal{X} \times \mathcal{U} \longrightarrow \mathcal{X} \tag{1}$$

where $\mathcal{X}$ *is the space of all possible states* $\mathbf{X}$ *and* $\mathcal{U}$ *is a space of all legal actions associated with the particularly mechanism* f*.*

Mechanisms like states, must be declared by an account and in many cases will have been declared along side specific local state variables X_i which the mechanism operates on. However, no such assumption will be made formally.

Definition 5. *For any mechanism* $f \in \mathcal{F}$*, for any agent* $a \in \mathcal{A}$ *there is a state dependent* ***action space*** *representing all legal actions for agent a given state* $\mathbf{X}$ *under mechanism* f*. A particular* ***action*** *is denoted* $u \in \mathcal{U}$*.*

Definition 6. *The set of all possible* ***transactions*** *is denoted* $\mathcal{T} = \mathcal{A} \times \mathcal{F} \times \mathcal{U}$ *where an element* $t \in \mathcal{T}$ *is defined* $t = (a, f, u)$*. In order for the transaction to be valid, agent a must have the right to perform the state update operation* $\mathbf{X}^+ = f(\mathbf{X}, u)$ *given the current state* $\mathbf{X}$*.*

As previously defined in the Ledger state, a sequence of transactions organized into a (partially) ordered list is denoted $\mathbf{T}$. The partial ordering may be considered without loss of generality by noting that such a partial ordering is defined precisely by the independence of the final output to the ordering of the list. This occurs when there is strong separation in the states, accounts and mechanisms involved in the transactions in $\mathbf{T}$.

Definition 7. *A* ***policy*** $P : \mathcal{X} \longrightarrow \mathcal{U}$ *is a state dependent strategy over a particular mechanism* $f \in \mathcal{F}$*. An agent* $a \in \mathcal{A}$ *is said to be using policy* P *over mechanism* $f \in \mathcal{F}$ *if it monitors the state* $\mathcal{X}$ *and broadcasts transaction* $t = (a, f, u)$ *associated with action* $u = P(x) \in \mathcal{U}$*.*

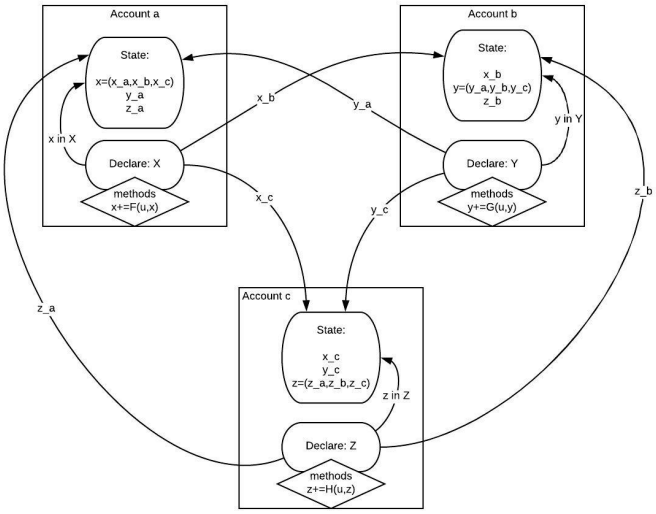


Figure 1. Consider three Addresses a, b, c which are accounts declaring state-mechanism pairs: $(x, F), (y, G), (z, H)$ respectively. Considering a, b, c as agents, they each have states x, y, z maintained by the account which declared them. Mechanisms F, G, H are available to all agents.

This definition allows for the case that no transactions are made because the conditions for a transaction under policy P are never met but it excludes the degenerate cases where there is no state for which a transaction will be generated. A practical consideration of this framing is the agents engagement level in monitoring the state. For the purpose of simple notation the sampling rate of the monitoring process is absorbed into the definition of P. During simulation. it is made explicit for tuning purposes.

Definition 8. *Consider ledger **State Transitions**; the Ledger State may be updated for any valid sequence of transactions* $\mathbf{T} = [\ldots, t, \ldots]$*, where* $t = (a, f, u)$ *is valid given* $\mathbf{X}$ *when the operation is applied. Without loss of generality, it is assumed all transactions are valid because invalid transactions are rejected.*

An atomic update is defined

$$\mathbf{X}^+ = f_t(\mathbf{X}, u_t) \text{ for any valid } (a_t, f_t, u_t) \text{ given } \mathbf{X} \tag{2}$$

where f_t is the mechanism used in transaction t and u_t is the action taken for transaction t. For a block defined by sequence of transactions $\mathbf{T}$, the state update is

$$\mathbf{X}^+ = f_N(f_{N-1}(\cdots f_1(f_0(\mathbf{X}, u_0), u_1) \cdots, u_{N-1}), u_N) \tag{3}$$

where the list of transactions $\mathbf{T}$ is indexed by $\{0, 1, \ldots, N\}$.

Definition 9. *The **Ledger Trajectory** is a sequence of ledger states* $\mathbf{L}(k) = \{\mathbf{X}(k), \mathbf{T}(k)\}$*, indexed by* $k = \{0, 1, \ldots, K\}$ *such that*

$$\mathbf{X}(k+1) = f_N(f_{N-1}(\cdots f_1(f_0(\mathbf{X}(k), u_0), u_1) \cdots, u_{N-1}), u_N) \tag{4}$$

for transactions $\mathbf{T}(k)$ *is indexed by* $\{0, 1, \ldots, N\}$.

Notation may be simplified by defining $\mathbf{F}_k$ to be the composition of the transactions in $\mathbf{T}(k)$ such that

$$\mathbf{X}(k+1) = \mathbf{F}_k(X(k)) \tag{5}$$

denoting the closed loop state update accounting implicitly for the actions $u = P((X))$.

At any time the most recent Ledger State may be denoted $\mathbf{L}(K) = \{\mathbf{X}(K), \mathbf{T}(K)$ where the integer K is the block height and $\mathbf{L}(0) = \{\mathbf{X}(0), \mathbf{T}(0)\}$ is the genesis block. The number of transactions is dependent on the block $N = N(t)$. Under this definition the **Blockchain** is characterized precisely by the trajectory of generalized dynamical system in conical form. As defined, the differential equations can be used to characterize the system with atomic transactions as the basic unit of time. Organizing transactions into blocks provides a means of testing block based logic.

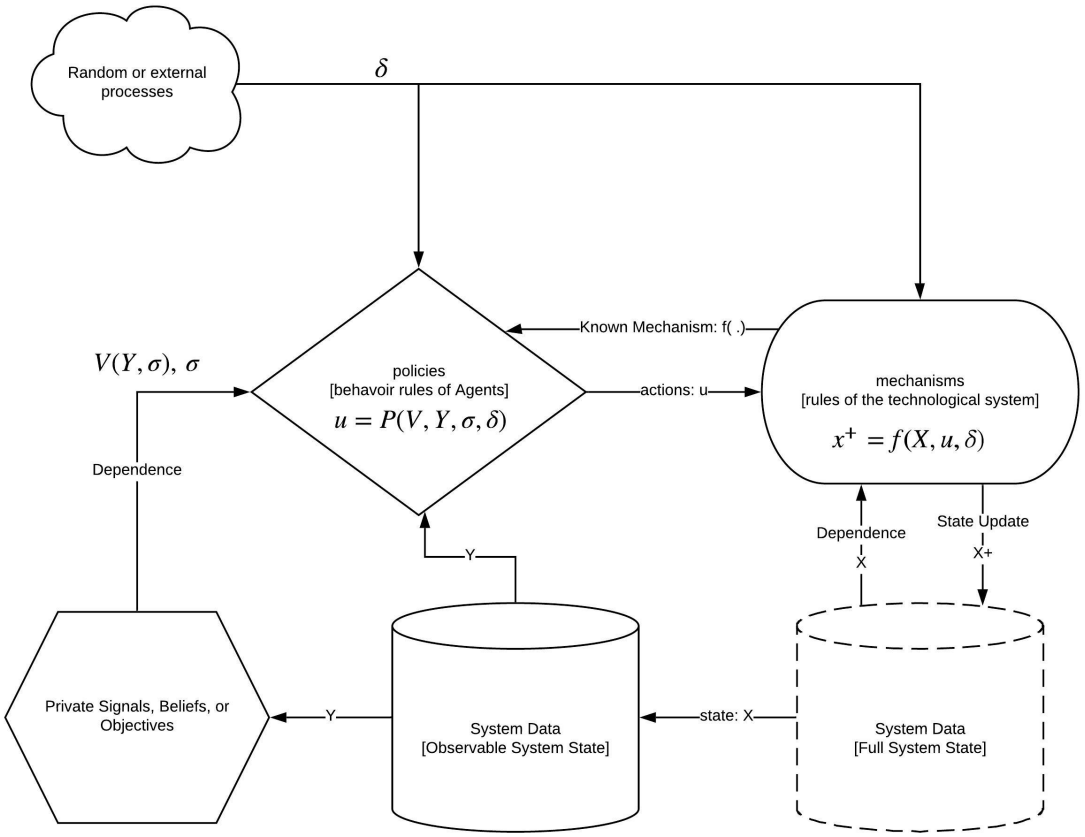


Figure 2. This is an expanded representation of the differential game accounting for a variety of factors beyond the states and actions including private signals σ, private goals $V(\cdot)$, a partially observable state $\mathbf{X}$ with an observable subspace $\mathbf{Y} \subseteq \mathbf{X}$, and some additional environmental stochastic processes δ drawn from potentially unknown distributions.

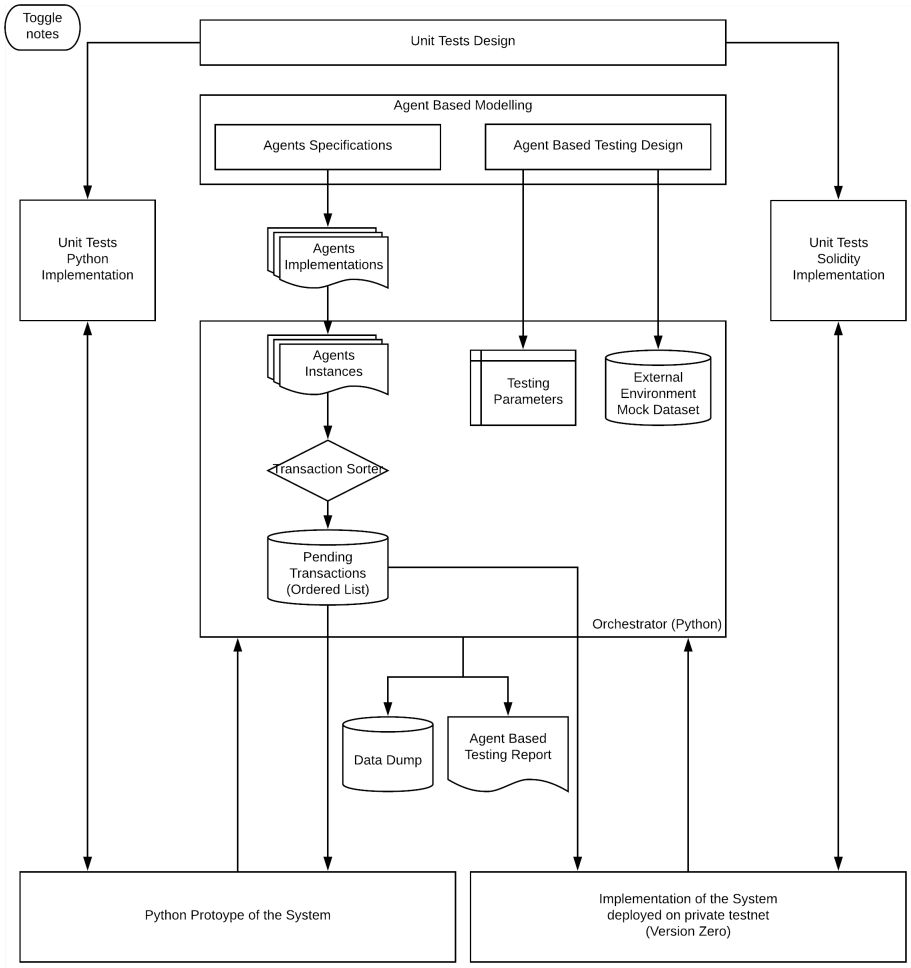


Figure 3. Our state model in turn allows us to represent differential equation based system designs and smart contract based implementations in the same paradigm and to further employ agent based modeling techniques to validate the degree to which an implemented system achieves the same system properties as a differential equations based design.